

Wah Thali Complete Website Handover Document

Website, Admin Panel, Workflows, Performance, Hosting, Integrations, and Commercial Handover Summary

Domain: wahthali.in

Next.js 16.2.11

React 19.2.4

Prisma 7.9

Supabase PostgreSQL

Razorpay

Meta WhatsApp

Prepared for: Wah Thali

Project: Wah Thali online food ordering website and admin operations platform

Prepared on: 07 September 2026

Production URL: <https://wahthali.in>

Commercial Summary

Initial discussed project cost: INR 10,000

Additional features increase: INR 2,000

Final project cost: INR 12,000

Amount already paid: INR 7,000

Remaining amount after handover/source-code delivery: INR 5,000

1. Executive Summary

Wah Thali is a production-shaped food ordering platform for fresh thalis, combos, bulk meals, customer accounts, order tracking, coupons, loyalty, online payments, WhatsApp communication, and admin-side restaurant operations.

The project includes a public customer website, secured admin pages, server-side order pricing, database-backed product/category/coupon/order/customer management, Razorpay payment handling, Meta WhatsApp OTP and notification support, and built-in WhatsApp retention automation.

Current verification result: production build, lint, and typecheck pass. Public health check on <https://wahthali.in/api/health> returns OK for domain, database, Supabase, payment environment, WhatsApp OTP, and WhatsApp notifications.

2. Website Map

Area	Routes	Purpose
Storefront	/ , /menu , /category/[slug]	Homepage/menu browsing, category navigation, search, filters, food cards, best sellers, offers, add-to-cart, wishlist.
Cart and Checkout	/cart , /checkout , /address	Cart review, coupon application, GST/delivery/packaging calculation, address/location capture, COD or Razorpay order placement.
Customer Account	/login , /account , /orders , /order/[orderNumber]/confirmed , /order/[orderNumber]/track	Password or WhatsApp OTP login, session handling, profile/order history, order confirmation, live order tracking.
Customer Engagement	/offers , /wishlist , /loyalty , /notifications	Coupons, saved dishes, reward tiers, customer notification preferences and order/offer messages.
Business Pages	/bulk-orders , /bulkorders , /corporate , /subscriptions , /support , /support/[topic] , /about	Bulk/corporate lead capture, subscription interest, help center, brand and business information.
Legal and SEO	/privacy-policy , /privacy-security , /terms-and-conditions , /refund-cancellation-policy , /delivery-policy , /data-deletion , /robots.txt , /sitemap.xml	Customer policies, privacy/data deletion, search-engine crawl support.

Admin	/admin, /admin/orders, /admin/inventory, /admin/categories, /admin/coupons, /admin/customers, /admin/automation, /admin/bulk- leads, /admin/reports, /admin/settings, /admin/access	Restaurant dashboard, operational controls, menu management, customer CRM, automation, reports, settings, staff permissions.
-------	--	--

3. High-Level System Diagram

```

Customer Browser
|
| Browse menu, cart, checkout, account, tracking
v
Next.js App Router on wahthali.in
|
+-- Public pages: menu, offers, policies, support
+-- Customer APIs: auth, OTP, profile, cart, orders, maps
+-- Admin pages: dashboard, orders, inventory, coupons, CRM, settings
|
v
Server Business Layer in src/lib
|
+-- Pricing, GST, delivery, coupons, store hours, state transitions
+-- Admin access, customer sessions, notifications, retention
|
+--> Supabase PostgreSQL via Prisma
+--> Supabase Storage for product/slider uploads
+--> Razorpay for online payment and refunds
+--> Meta WhatsApp Cloud API for OTP/order/retention messages
+--> Google Maps APIs for geocoding and delivery location
+--> Optional n8n signed webhook automation

```

4. Customer Workflow Design

```

Open wahthali.in
-> Select/check delivery location
-> Browse categories/menu
-> Search/filter/sort dishes
-> Open dish details when variants/addons are required
-> Add item to cart or wishlist
-> View cart
-> Apply coupon
-> Server recalculates subtotal, discount, delivery, GST, grand total
-> Checkout
-> Enter customer, receiver, address, PIN/GPS, note
-> Select COD or online payment
-> Place order
    -> COD: order enters NEW or CONFIRMED depending on admin auto-accept setting
    -> Razorpay: order enters PENDING_PAYMENT, payment is verified, then order enters NEW/CONFIRMED
-> Customer sees order tracker
-> Admin updates preparation/delivery status
-> Customer receives configured WhatsApp updates

```

-> Delivered order can trigger retention/review journey

5. Admin Workflow Design

Admin/Staff Login

- > AdminAccessGate validates customer session
- > Server checks role/permissions
- > Allowed module opens

Daily operations:

Dashboard

- > View sales, revenue, orders, low stock, unavailable items

Orders

- > See incoming orders
- > Accept / prepare / pack / ready / dispatch / deliver / decline
- > Print bill
- > Trigger customer/owner WhatsApp notifications
- > Initiate Razorpay refund after cancelled online-paid order

Inventory

- > Add/edit dishes, pricing, images, variants, addons, stock, availability

Categories

- > Manage parent/subcategory structure, images, offers, visibility, order

Coupons

- > Add/edit/disable coupon rules and send WhatsApp coupon notifications

Customers

- > View customer profiles, tags, order history, LTV, reward tier

Automation

- > Configure retention journeys, templates, coupons, test sends, process due messages

Bulk Leads

- > Review party/corporate/bulk order enquiries

Reports

- > Sales/tax/performance reporting and GSTR-1 API

Settings

- > Store mode, hours, delivery fees, distance slabs, payments, sounds, contact numbers

Staff Access

- > Grant/admin/manager/staff roles and module permissions

6. Admin Pages and Handling

Page	Main Features	Operational Handling
/admin	Dashboard metrics, quick action queue, product availability table.	Use as the daily start screen. Check low stock, active orders, unavailable items, and quick menu controls.
/admin/orders	Kitchen board, 15-second refresh, browser notifications, new/cancelled order alerts, filters, status controls, ETA/location notes, printable bill, reset order history.	Move orders through valid statuses only. Decline with reason when required. For Razorpay orders, refund is available after cancellation if payment is captured.

/admin/inventory	Product CRUD, price/original price, tax, dietary type, availability, bestseller, category, images, variants, addons, stock and reorder values.	Keep unavailable dishes offline instead of deleting. Upload images to Supabase Storage or use public paths.
/admin/categories	Category CRUD, parent-child hierarchy, sort order, visibility, category images, category-level offers.	Use parent categories for broad menu sections and subcategories for detailed grouping.
/admin/coupons	Coupon CRUD, fixed/percent discounts, min order, max discount, audience rules, product/category restrictions, channels, fulfillment modes, WhatsApp coupon notify.	Check start/end dates and redemption limits before publishing customer offers.
/admin/customers	Customer cards, profile details, birthday/anniversary, VIP/tagging, order history, LTV, reward tier, WhatsApp actions.	Use for CRM, loyalty segmentation, and support follow-up.
/admin/automation	WhatsApp retention configuration, templates, campaign stats, scheduled/sent/failed messages, dormant scheduling, test template sends.	Seed/test templates first. Process due messages only when templates and customer opt-ins are confirmed.
/admin/bulk-leads	Bulk/corporate enquiries, lead settings, notification configuration.	Monitor new party/corporate leads and follow up from WhatsApp/phone.
/admin/reports	Sales and performance reporting, tax/GSTR-1 endpoint support.	Use date filters and export/report data for accounting review.
/admin/settings	Store mode, hours, messages, prep minutes, payment toggles, delivery fee mode, PIN/radius serviceability, GST, packaging, slider banners, home dish categories, alert sounds.	Update operational settings without code changes. Verify after saving because settings affect checkout immediately.
/admin/access	Staff role and permission management.	Admin can grant Manager or Staff access. Staff starts with order-only permission unless expanded.

7. Feature Inventory

Customer Features

- Responsive food-ordering website for mobile and desktop.

- Menu categories, nested category navigation, category images, search, filters, and price sorting.
- Dish cards with images, ratings, bestseller labels, offer pricing, vegetarian/non-vegetarian mark, availability state.
- Dish detail handling for variants and addons.
- Cart stored per guest/customer session, coupon handling, free-delivery progress, GST, delivery and packaging calculation.
- Checkout with delivery address, PIN code, GPS location, restaurant note, receiver details, WhatsApp marketing opt-in, COD and Razorpay modes.
- Order confirmation and live tracking by order number.
- Customer login by password or WhatsApp OTP, password reset, session endpoint, profile and saved addresses.
- Wishlist, offers page, loyalty page, notifications, support center, bulk-order and corporate enquiry pages.
- Legal pages for terms, privacy, refund/cancellation, delivery/pickup, and data deletion.

Business/Admin Features

- Permission-protected admin dashboard and admin APIs.
- Roles: Admin, Manager, Staff; permissions: dashboard, orders, inventory, categories, coupons, customers, automation, leads, reports, settings, access.
- Server-side order creation with database product lookup, availability checks, modifier checks, coupon eligibility, GST, delivery distance/radius logic, minimum order validation, and kitchen capacity checks.
- Admin order status machine prevents invalid transitions.
- Order cancellation restores inventory stock.
- Razorpay order creation, payment signature verification, webhook reconciliation, and refund initiation.
- WhatsApp OTP, customer order notifications, owner alerts, coupon notifications, inbound message capture, STOP opt-out handling, feedback/review retention flow.
- Settings stored in database through **BusinessSetting**, allowing operational changes without redeploy.
- Activity logs and webhook event persistence for auditability.

8. Database Design Summary

The Prisma PostgreSQL schema covers the following main groups:

Group	Models
Staff access	User, Role, Permission, UserRole, RolePermission
Customers	Customer, CustomerAddress, CustomerTag, CustomerNote, CustomerOtp, CustomerPasswordResetToken
Menu	Category, Product, ProductImage, ProductVariantOption, ProductAddon, InventoryItem
Cart and Orders	Cart, CartItem, Order, OrderItem, OrderStatusHistory, Payment
Marketing and CRM	Coupon, CouponRedemption, LoyaltyAccount, Lead, LeadStageHistory, Review, SupportTicket

Automation	WhatsAppMessage, WhatsAppTemplateRegistry, RetentionCampaign, RetentionMessage, AutomationRule, AutomationRun
Operations	BusinessSetting, IntegrationSetting, ActivityEvent, AuditLog, WebhookEvent, IdempotencyKey

9. Order Status Workflow

Online payment path:

PENDING_PAYMENT -> NEW or CONFIRMED -> PREPARING -> PACKED -> OUT_FOR_DELIVERY -> DELIVERED

PENDING_PAYMENT -> CANCELLED when payment fails/cancels

COD delivery path:

NEW -> CONFIRMED -> PREPARING -> PACKED -> OUT_FOR_DELIVERY -> DELIVERED

NEW / CONFIRMED / PREPARING -> CANCELLED if restaurant declines

Pickup path:

NEW -> CONFIRMED -> PREPARING -> PACKED -> READY_FOR_PICKUP -> DELIVERED

Terminal statuses:

DELIVERED and CANCELLED cannot be moved further by normal admin controls.

10. Hosting, Domain, and Environment

Item	Status / Detail
Production domain	https://wahthali.in
WWW handling	https://www.wahthali.in redirects/serves the same website.
Current health endpoint	https://wahthali.in/api/health returned HTTP 200 and <code>ok: true</code> on 07 September 2026.
Deployment config present	<code>netlify.toml</code> uses Node 22 and build command <code>npm run build</code> .
Documented hosting note	Project docs mention GoDaddy Node.js Hosting currently used for the live domain, with Vercel/Netlify as target-style deployment options.
Database	Supabase PostgreSQL through Prisma. Local and production health indicate <code>DATABASE_URL</code> is configured.
Storage	Supabase Storage bucket expected: <code>wah-thali-assets</code> .
Secret handling	Secrets should remain only in local <code>.env.local</code> and hosting provider environment variables. They must

	not be committed into source code.
--	------------------------------------

Environment Groups

- Core: `NEXT_PUBLIC_SITE_URL`, `DATABASE_URL`, `DIRECT_URL`
- Supabase: `SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_URL`, publishable/anon key, service role key, storage bucket
- Admin: `ADMIN_MOBILES`, `ADMIN_EMAILS`, `ADMIN_BOOTSTRAP_PASSWORD`, `ADMIN_BOOTSTRAP_NAME`
- Razorpay: key id, key secret, webhook secret
- WhatsApp: phone number id, access token, verify token, OTP template, order/coupon/lead templates, language, country code, Graph API version
- Automation: `N8N_SHARED_SECRET`, `CRON_SECRET`
- Maps: `GOOGLE_MAPS_API_KEY`
- Email: SMTP host, port, secure flag, user, password, from address

Local audit showed SMTP host/user/password are not currently filled locally. Other listed production-critical groups were present locally. Values are intentionally not printed in this document.

11. Integrations

Integration	Implemented Handling	Required Operational Action
Supabase PostgreSQL	Prisma models, product/category/coupon/order/customer queries, settings, logs, webhook events.	Keep backups, review migrations before applying, keep service role server-only.
Supabase Storage	Upload endpoints for assets and admin slider/product images.	Confirm bucket exists and storage policies allow intended server uploads.
Razorpay	Create Razorpay orders, verify checkout payment signature, webhook reconciliation, refund initiation.	Configure webhook URL https://wahthali.in/api/webhooks/razorpay and subscribe to payment/refund events.
Meta WhatsApp Cloud API	OTP login/signup, order notifications, owner alerts, coupon notifications, retention messages, inbound reply storage.	All templates must be approved in Meta Business Manager and names must match environment variables.
Google Maps	Autocomplete, geocode, reverse geocode, GPS/delivery radius support.	Restrict API key by domain/API and monitor quota.
n8n	Optional signed webhook route with HMAC shared secret.	Use only signed requests with <code>x-wah-signature</code> .
SMTP	Password reset/email support exists but local SMTP details are not filled.	Add SMTP credentials in production if email delivery is required.

12. Security and Access Control

- Admin pages are wrapped by an admin access gate and server page permission checks.
- Admin APIs require signed-in identity headers supplied by the admin client session and checked by server permission helpers.
- Bootstrap admin access is controlled by `ADMIN_MOBILES`, `ADMIN_EMAILS`, and `ADMIN_BOOTSTRAP_PASSWORD`.
- Razorpay webhook signatures are verified using HMAC and timing-safe comparison.
- n8n webhook route is designed for HMAC shared-secret validation.
- Order pricing is recalculated server-side; browser totals are not trusted for final order creation.
- Webhook events are persisted with provider/event IDs to reduce duplicate processing risk.
- Recommended post-handover hardening: apply Supabase RLS policies, add rate limiting for OTP/login/order endpoints, rotate any shared handover secrets, and review public order lookup visibility.

13. Performance Review

Build-Time Performance

Check	Result
Production build	Passed. Build compiled successfully in 11.5s; TypeScript stage finished in 19.3s; 75 static pages generated in 6.1s.
Route rendering	Public homepage, about/legal/support/offers pages are static or ISR where possible. Admin and APIs are dynamic.
Storefront cache	Public home/menu/cart/offers/wishlist data loaders use Next cache with 60-second revalidation tags.
Images	Food/category assets are delivered from Supabase Storage and local public assets; menu cards use lazy image loading in client components.

Public Domain Spot Check

URL	Status	Total Time	Start Transfer	Download Size
https://wahthali.in/	200	2.4059s	1.8543s	248,558 bytes
https://wahthali.in/menu	200	1.9994s	1.1524s	783,964 bytes
https://wahthali.in/api/health	200	2.3681s	2.3679s	2,117 bytes

These are simple curl-based timing checks from the development machine, not a full Lighthouse lab report. For final production sign-off, run Lighthouse/PageSpeed on mobile and desktop after DNS, hosting, and caching are stable.

Performance Recommendations

- Keep product images compressed and correctly sized before uploading.
- Use CDN-backed image delivery for Supabase Storage assets.
- Monitor API response time for `/api/health`, `/api/orders`, and admin order refresh.
- Keep storefront data cache revalidation at 60 seconds or adjust only if menu updates need instant propagation.
- Add a Lighthouse report during final deployment acceptance to record Core Web Vitals.

14. Verification Status

Command / Check	Status	Notes
<code>npm run lint</code>	Passed	No ESLint errors reported.
<code>npm run typecheck</code>	Passed	TypeScript completed with no reported errors.
<code>npm run build</code>	Passed	Next.js production build completed successfully.
<code>npm run test</code>	Needs update	4 test files ran; 3 passed; <code>src/lib/pricing.test.ts</code> has 7 failing assertions because pricing tests reference product IDs while the local fallback product catalog is currently empty.
Public health check	Passed	<code>https://wahthali.in/api/health</code> returned <code>ok: true</code> .
Git worktree before document generation	Clean	No pending source-code changes before creating this handover PDF source.

15. Known Handover Notes

- Update pricing unit tests to use seeded/factory products or restore representative fallback products in `src/lib/data.ts`.
- SMTP credentials are not filled locally; password reset email needs production SMTP configuration if email flow is required.
- Verify all Meta WhatsApp template names in production exactly match approved templates.
- Run at least one real Razorpay test payment and one refund test using the production/test Razorpay dashboard before accepting online payment operations.
- Apply and document Supabase RLS policies if direct client-side Supabase access is expanded in future.
- Rotate any keys shared during handover once the client confirms ownership of hosting, database, payment, WhatsApp, and domain accounts.

16. Source Code Handover Checklist

1. Provide complete project folder/source repository.
2. Confirm production deployment account ownership and collaborator access.

3. Confirm domain DNS access for wahthali.in and www.wahthali.in.
4. Confirm Supabase project ownership, database backup, and storage bucket access.
5. Confirm Razorpay account ownership and webhook configuration.
6. Confirm Meta Business/WhatsApp account ownership, approved templates, webhook verification token, and access token management.
7. Confirm admin bootstrap mobile/email/password and immediately change/rotate after handover.
8. Run final deployment checks: health endpoint, customer login, add-to-cart, COD order, Razorpay test order, admin status update, WhatsApp OTP, WhatsApp order message, and image upload.
9. Settle remaining commercial amount after handover/source-code delivery as agreed.

17. Commercial and Payment Summary

Description	Amount
Original project amount first discussed	INR 10,000
Additional amount for increased features and scope	INR 2,000
Final total project cost	INR 12,000
Amount already paid	INR 7,000
Remaining amount payable after handover and source-code delivery	INR 5,000

18. Final Acceptance Statement

This document summarizes the current Wah Thali website and admin platform, including customer workflows, admin handling, database structure, domain/hosting status, integrations, performance notes, verification status, and handover checklist as of 07 September 2026.

After the source code, deployment access, environment ownership, and final checks are transferred, the remaining INR 5,000 is payable as the final handover balance.